

Spout Finance beta review

The interface said my order was signed. The chain never saw it.

I tested Spout Finance on Solana devnet on 18 and 19 September 2026. I funded a new embedded wallet with 5 SOL and 20 USDC, completed the identity flow, and tried to buy \$20 of NVDA twice. Neither attempt became a buy transaction by my final check.

The first attempt returned `Blockhash not found from /api/orders/submit`. The second built an order but never issued the submit request. Spout showed the same green signed/submitting state in both cases and did not record either failure in Activity or Transaction History. That false-success state is the most urgent issue in this report. The rest of the review looks at the controls and disclosures that become important before this product handles real tokenized equities.

Test detail	Value
Build tested	beta.spout.finance, Solana devnet, v1.0.0 (5d92f4c)
Wallet	DiiNsocoaZ9SeSk3DgxJH8gdGZynFcc9ydvRYMzuN69J
Access card	No. 201
Funding	5 SOL and 20 USDC devnet
Test dates	18–19 September 2026
Assets used	Valueless devnet assets only

Findings

Priority	Finding	Evidence status
Blocker	The order UI reported a signed/submitting state after two different failures; neither buy appeared on-chain by my final check	Direct browser observation, reproduced twice
High, pre-mainnet	The NVDA mint has a permanent delegate, defaults new token accounts to frozen, and has a freeze authority, but the product does not explain these controls	Confirmed on-chain
High, pre-mainnet	Orders and Vault were upgradeable through the same on-curve authority during my check	Confirmed on-chain for the programs listed below
High disclosure risk	The interface displayed <code>Reserves 100.2%</code> as a fixed client string and provided no path to an attestation	Confirmed in the client bundle; devnet reserve account checked separately
High design question	At the displayed 50% maximum LTV, a 2.0x position implies Health Factor 1.00 if Spout uses the same 50% as the liquidation threshold	Derived, not tested with a live position
Medium	<code>0% interest</code> is technically different from the displayed Borrow Cost, but the prominent claim hides the economic cost in capped upside	Confirmed UI wording; accounting interpretation needs clearer disclosure
Resolved during testing	The production CSP blocked Spout's configured devnet RPC on 18 September	Confirmed during testing; no longer present in the 19 September check

Priority	Finding	Evidence status
Medium	NVDA market data disagreed between the table and asset page	Direct browser observation
Medium	The leverage slider was not keyboard operable; light-mode labels measured 3.32:1 contrast	Measured in the tested build
Medium	Terms and Privacy controls did not open a document or route	Direct browser observation
Low to medium	The tested build had amount-entry, rounding, minimum-order, responsive-layout, and stale Sell-panel defects	Direct browser observation

Scope and limitations

Everything in this report concerns devnet. It does not establish how Spout will operate on mainnet.

I did not complete a purchase. I therefore could not test borrowing, repayment, selling, collateral release, liquidation, or Earn end to end. The leverage section is a calculation from displayed parameters, not a claim that I triggered liquidation.

I did not verify brokerage ownership, legal title to shares, custodian records, real-world reserve attestations, or investment performance. A devnet token balance and a devnet USDC account cannot prove or disprove off-chain backing.

I used the product UI for the user flows and public, read-only RPC calls for chain inspection. I did not attempt an exploit, submit a hand-built transaction, or bypass a product control. The evidence directory contains public RPC and live-client snapshots captured after the test dates. Those snapshots support facts that remain publicly reproducible; they do not replace the historical Network-panel observations described below.

1. Signed did not mean submitted

I attempted to buy \$20 of NVDA. Privy opened, I signed, and Spout displayed:

Order signed

Submitting it to Solana now. This can take a moment.

No position appeared, my USDC balance did not change, and the interface showed no error.

The Network panel recorded this sequence on the first attempt:

```
POST /api/orders/buy      200
POST /api/orders/submit  400
{"error":"Simulation failed. \nMessage: Transaction simulation failed: Blockhash not found."}
```

This response means the blockhash in the submitted transaction was no longer valid when simulation ran. I did not capture when the application fetched that blockhash, so I cannot determine which step consumed its validity window or whether signing latency alone caused it.

The second attempt followed a different path:

```
POST /api/orders/buy      200
No following /api/orders/submit request
```

Spout showed the same green signed/submitting state. Because the submit request never appeared, the first attempt's blockhash error does not explain the second attempt.

The common problem is the state model. The interface treated wallet signature as success before it knew whether the backend had accepted, broadcast, confirmed, or settled the transaction. Neither attempt appeared in Portfolio Activity or Transaction History. By my final on-chain check, the wallet still had no buy transaction or stock-token balance.

The only new signature was:

```
5gj3NrumwWU8voNErhmpi6dndgQ4jggQDWZN9534xa6zdo6mtXJTPaVvynUvqnPLzqVUv9wqmk9MJEyaaeP7bzpr
```

Its logs contain `CreateIdentityForUser` and `SetVerified` under Spout's KYC program. It is not a buy. The decoded transaction is saved in `evidence/kyc-transaction.json`.

What I would change

1. Treat wallet signature as its own intermediate state, not success.
2. Show separate states for signed, submitted, confirmed, and settled.
3. Track `lastValidBlockHeight`. After confirming that the original transaction did not land and its blockhash is no longer valid, rebuild it with a fresh blockhash and request a new signature.
4. Make that recovery path idempotent so it cannot create a duplicate order.
5. Put pending and failed attempts in Activity, including a reason code and transaction signature when one exists.
6. Tell the user plainly when nothing was charged.

2. Token controls are appropriate, but invisible

The NVDA asset uses a Token-2022 mint:

```
Mint
FkzjAJX584L1LncQb8AKSnGhc5sLvZjPGCgZLGpeHyTH

Permanent delegate
28bLmwghfAyZLCXpRZHpxa7uefN1TupnCftXfTqHzVJm

Default account state
frozen

Freeze authority
GDBgF4AdFnK3ku8e7euZZLUag3JU9zDzbGktXTrzfMtE
```

The current parsed mint account is included in `evidence/public-rpc-snapshot.json`.

These fields matter:

- A permanent delegate can authorize transfers or burns from token accounts for that mint without the token holder signing that instruction.
- The token holder cannot remove the mint-level permanent delegate from their own account. The mint authority can rotate the permanent delegate through Token-2022's authority flow.
- New token accounts default to frozen and require an authorized thaw before normal transfers.
- The freeze authority can freeze eligible token accounts.

These controls are not automatically a vulnerability. Regulated securities may need transfer restrictions, sanctions controls, court-order handling, and recovery procedures. The problem is that the interface uses simple ownership language without explaining the issuer and program controls attached to that token.

Spout should add a Token controls section to each asset page. It should name the mint, token program, permanent delegate, default state, freeze authority, and the entity or program responsible for each authority. A short explanation would turn a hidden trust assumption into useful disclosure.

3. The devnet upgrade model concentrates control

I independently rechecked three programs directly referenced by the tested client:

Program	Address	Upgrade authority at check time
Orders	SPoRXgsB4gWZmWPwyndoRWQrmZXKUc7o7oPMdkkGRcG	7N31cE8BRTpyAVDeczkutP4EnGdQJLSQm4q3SJ6aYWEp
Vault	spvaDgABYdpFKyatqo4Jr3nfwvVgWF5BbFxKFkzN3Am	7N31cE8BRTpyAVDeczkutP4EnGdQJLSQm4q3SJ6aYWEp
KYC identity	SKYCVrkX3mQaHwZcLrUtuVzii43kma7kBUim5MaQm6k	BD29wQ5Tj7b1MEFqquRxASsTqoN38oCrjxpU4riTZx7C

The Orders and Vault programs shared the same direct authority address. The authority addresses above were on-curve and their accounts were System Program-owned with zero data at the time of the snapshot. This rules out an on-chain multisig or governance PDA as the direct authority. It does not reveal whether signing is protected by an HSM, MPC system, custodian, off-chain approval policy, or operational delay.

This is normal for a beta on devnet. I am not presenting it as an exploitable production vulnerability. It is a pre-mainnet governance and operational-security question because an upgrade authority can replace program logic. Where a token authority is exercised through an upgradeable program, its behavior also depends on that program's deployed logic and upgrade process.

Before mainnet, I would publish:

- whether upgrade authority will move to a multisig and its signing threshold;
- whether upgrades will have a timelock or public notice period;
- whether any program will become immutable;
- whether Orders and Vault will retain a shared authority;
- the emergency-upgrade and key-compromise procedure.

The raw program and ProgramData accounts are included in [evidence/public-rpc-snapshot.json](#).

4. Reserves 100.2% looked measured, but was static

Every tested order panel displayed:

Held 1:1 at Alpaca Securities · Reserves 100.2%

The footer also displayed:

US-regulated custody | Onchain proof of reserves

I found `Reserves 100.2%` as a literal string in the client bundle rather than a value fetched or computed from a visible reserve feed. The proof-of-reserves label did not link to an attestation, account, methodology, or timestamp.

The client also declared this devnet USDC account:

ADZRYU7F5t4vzhBCihr9faNEYPbGAHYCiC8c3mkZKR2d

Its token balance was 0 USDC when checked. That does not show that Spout lacks real-world reserves. A devnet USDC account cannot establish the number of shares held by a broker or the legal claim represented by a stock token. It only shows that the displayed 100.2% was not explained by the devnet account shown in the same client configuration.

The defensible finding is about disclosure: the beta presented a precise reserve ratio without a user-verifiable path from the number to its source.

A reserves page should show, per ticker:

- token supply;
- custodian-held shares or the relevant attested quantity;
- the resulting ratio and formula;
- attestation provider and scope;
- last-updated time;
- links to the on-chain accounts involved.

If that system is not ready on devnet, [Reserve reporting launches with mainnet](#) is better than a fixed percentage.

The live-client string and current CSP snapshot are in [evidence/live-client-snapshot.json](#). This file was captured after my original test and is labeled accordingly.

5. Maximum leverage may coincide with the displayed liquidation boundary

I could not open a leveraged position, so this section is a design question derived from Spout's displayed numbers.

The tested interface said:

- At 2.0x, the user puts up half and borrows half.
- Each \$10,000 of eligible stock supports borrowing up to \$5,000, which implies a 50% maximum LTV.
- A Health Factor of 1.00 is the point at which collateral can be liquidated.

For user capital C , a 2.0x position would have debt C and collateral value $2C$. If Spout also uses 50% as the liquidation threshold, the standard health-factor calculation gives:

$$\begin{aligned} \text{LTV} &= C / 2C = 50\% \\ \text{Health Factor} &= (2C * 0.50) / C = 1.00 \end{aligned}$$

That assumption needs confirmation. Maximum borrowing LTV and liquidation threshold are not always the same parameter, and I did not inspect a live position or the liquidation program. The safe conclusion is that the current preview does not give the user enough information to resolve the risk.

The Buy panel should display projected Health Factor and liquidation price as leverage changes. Spout should also publish the exact LTV, liquidation-threshold, oracle, market-closure, and gap-risk rules used by the protocol.

6. 0% interest needs the cost beside the claim

The interface made these statements at the same time:

Surface	Wording
Banner	0% Interest on borrowing. Always, no matter the market conditions.

Surface	Wording
Trade table	Borrow Cost values up to 0.86%/yr in the tested data
Ask Spout	There is no interest charge for borrowers.
Borrow table	Column labeled Interest Rate

This is not necessarily an accounting contradiction. The bundle's tooltip described Borrow Cost as annualized upside capped by a covered call, not recurring cash interest. A borrower may pay no interest and still bear an economic cost through foregone gains above the option strike.

The problem is prominence. The absolute 0% claim was persistent, while the explanation of capped upside was placed in a tooltip. Borrow Cost and Interest Rate also appeared to label related concepts inconsistently.

A clearer version would be:

0% cash interest. Your borrowing cost is capped upside.

A covered call funds the loan. Spout shows the estimated annualized cost for each asset.

The product should also disclose how the estimate depends on strike selection, expiry, volatility, assignment, roll timing, and fees. Those parameters determine whether the displayed annualized figure resembles the borrower's realized outcome.

7. The CSP regression was no longer present on 19 September

On 18 September, the app displayed a 0 USDC balance even though the funded wallet held 20 devnet USDC. The browser reported that its Content Security Policy blocked <https://api.devnet.solana.com>, the RPC configured by the client.

That one policy error explained the failed balance, price-account, and KYC-account reads I saw in that session.

By 19 September, the HTTP and WebSocket devnet RPC origins were present in `connect-src`. Balance, price, and KYC reads recovered. The current header in `evidence/live-client-snapshot.json` still contains both origins.

This was no longer an active blocker in my 19 September check. A deployment check should fail when a configured RPC origin is absent from CSP, or Spout could proxy RPC traffic through a same-origin endpoint.

8. Market data disagreed between screens

During the test, NVDA showed a \$5.36T market cap in the trade table and \$2.59T on its detail page. At the displayed \$221.39 share price, those values implied very different share counts.

The NVDA fundamentals grid also duplicated values across fields with incompatible units:

EPS	\$6.31
ROE	\$6.31
Dividend Yield	0.41%
EPS (TTM)	0.41%

ROE should not be formatted as dollars, and EPS should not be formatted as a percentage. GOOG displayed distinct values with appropriate units, so the issue appeared asset-specific rather than a universal field mapping failure.

For a product using market data to support lending decisions, every displayed value should carry a source and freshness timestamp. Field-level schema validation should reject incompatible units before publication.

9. Financial controls need keyboard and contrast fixes

The leverage control was a custom `div` with `role="slider"`, but it had `tabindex="-1"`. Programmatic focus followed by four ArrowRight presses left `aria-valuenow` unchanged. Only dragging the thumb changed the value.

That makes a debt-setting control inaccessible to keyboard users. A native range input or a slider component with correct focus, arrow-key behavior, and ARIA state would fix it.

In light mode, 12px ticker labels used `rgb(141, 141, 141)` on white. I measured a 3.32:1 contrast ratio. WCAG AA requires 4.5:1 for normal-size text.

At a 1024px viewport, the asset-detail `View` control also sat underneath the order panel. At 375px, the borrow-cost text overflowed its container by 11px and clipped `/yr` from each row.

10. Legal and notification controls were incomplete

Terms of Service and Privacy Policy were rendered as buttons in the tested build. Clicking them did not open a route, modal, document, or new tab.

Settings also offered enabled notification toggles, including a Health Factor alert, without showing an email address, phone number, push permission, or delivery channel. I could not determine where a liquidation warning would arrive.

Before mainnet, legal links should resolve to versioned documents and critical-risk alerts should show their destination, opt-in status, and last successful delivery test.

11. Smaller interaction defects

These did not carry the same risk as the findings above, but they made the product harder to trust:

- Switching from Buy to Sell carried the Buy amount into a zero-holdings Sell panel and displayed a nonzero amount the user would receive.
- Switching between dollar and share units changed the economic amount because the share display rounded too early.
- On macOS, Command+A followed by typing appended to the existing amount or search text instead of replacing it.
- A \$1 order remained enabled even though the client configuration declared a \$2 minimum.
- The Privy modal mixed decimal separators.
- Ask Spout did not submit with Enter.
- The product tour described Earn flows while the Earn page said `coming soon`.
- Export CSV remained enabled with no transactions.

The fix is to keep monetary precision in the model, round only for display, reset state when trade direction changes, respect native text-selection behavior, and validate minimums before opening the wallet flow.

How I read the economics

Ask Spout described a model in which covered-call premiums fund lender yield, Spout takes a protocol fee, and borrowers pay no recurring cash interest. I did not independently verify the option execution, fee split, or settlement path because I could not complete a purchase.

The model is plausible, but users need to understand which risk moves where. A covered call caps upside and does not remove most downside risk. Under the flow Ask Spout described, assignment followed by repurchase above the prior strike can reduce the number of shares bought with residual proceeds. If Junior absorbs the book's first losses, it can also carry concentrated downside exposure. I could not verify either implementation in a live position.

The missing parameters matter more than the slogan:

- strike-selection method;
- option tenor and roll cadence;
- assignment and repurchase mechanics;
- periods between option cycles;
- realized borrower cost in dollars;
- lender waterfall and first-loss limits;
- oracle freshness rules;
- liquidation behavior while US markets are closed.

Publishing those parameters and a reproducible historical analysis would let users judge the trade-off instead of inferring it from 0% interest.

What I would fix first

1. Replace the false-success order state with an explicit signed, submitted, confirmed, settled, or failed state.
2. Log pending and failed orders in Activity and give the user a reason code.
3. Disclose Token-2022 controls on every asset page.
4. Link the reserve claim to an attestation, or remove the percentage until one exists.
5. Publish the mainnet authority and upgrade plan.
6. Show Health Factor and liquidation price before a leveraged order is signed.
7. Put capped-upside cost beside the 0% cash-interest claim.
8. Add deployment tests for CSP, market-data schemas, keyboard controls, and responsive clipping.
9. Wire Terms, Privacy, and alert-delivery settings before public launch.

Overall assessment

Spout's product thesis is worth pursuing. The interface explains a difficult options-backed lending model better than most early DeFi products I have tested. The Token-2022 controls are also consistent with a restricted-asset design, although this run did not establish their exact PDA derivations or operational governance.

The current beta still asks the user to trust important states that are not verifiable from the interface. My order was the clearest example: the wallet signature looked like success even though one submission failed and the other never started. The same gap appears in the fixed reserve percentage, the hidden token controls, and the leverage preview.

The strongest improvement would be to make every important claim traceable. An order should have a state and reference. A reserve percentage should have an attestation and timestamp. A token should disclose its authorities. A leveraged position should show its liquidation assumptions before the user signs.

Reproduction notes

Public snapshots captured after the original test are included in [evidence/](#):

- [public-rpc-snapshot.json](#) contains wallet signatures, wallet balances, parsed NVDA mint data, the declared devnet reserve-account balance, and upgradeable-program account data.
- [kyc-transaction.json](#) contains the full decoded transaction and logs for the KYC signature.
- [live-client-snapshot.json](#) contains the live CSP header, client-bundle hashes, and short contexts for the relevant literal strings. Its timestamp is later than the original test and is not presented as a historical capture.

The core public RPC checks can be rerun with:

```
RPC=https://api.devnet.solana.com

curl -s -X POST "$RPC" -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"getSignaturesForAddress","params":
  [{"DiiNsocoaZ9SeSk3DgxJH8gdGZynFcc9ydvRYMzuN69J"}, {"limit":10}]}'

curl -s -X POST "$RPC" -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"getAccountInfo","params":["FkzjAJX584L1LncQb8AKSnGhc5sLvZjPGCgZLGpeHyTH",
  {"encoding":"jsonParsed"}]}'

curl -s -X POST "$RPC" -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"getTokenAccountBalance","params":
  [{"ADZRYU7F5t4vzhBCihr9faNEYPbGAHYCiC8c3mkZKR2d"}]}'

curl -s -X POST "$RPC" -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"getAccountInfo","params":["SPoRXgsB4gWZmWPwyndoRWQrmZXKUc7o7oPMdkkGRcG",
  {"encoding":"base64"}]}'
```

Historical UI and Network-panel observations are labeled separately from the later public snapshots.